



Tecnun
Universidad
de Navarra

ESCUELA DE INGENIERÍA
INGENIARITZA ESKOLA
SCHOOL OF ENGINEERING

C++ Laboratory Lessons

Lesson N° 10

C++
PARA INGENIEROS

Computer Programming II

Dr. Paul Bustamante / Dr. Jesús Paredes

OUTLINE

Contents

1.1. Exercise 1: Inheritance: Class Employee.....	1
1.2. Exercise 2: Constructor and destructor execution sequences.....	2
1.3. Exercise 3: Calling the constructors	2
1.4. Exercise 4: Adding more variables to the class.....	3
1.5. Exercise 5: Additional work	3
1.5.1. Adding even more variables	3
1.5.2. Form public to protected.....	3
1.5.3. Form protected to private	3
1.5.4. Dynamic structure	4
1.6. Exercise 5: Vehicle management.....	4
1.7. Additional work.....	5
1.7.1. Objects within structures	5
1.7.2. Array of structures with objects within the structure	5
1.7.3. Dynamic array of structures with objects within the structure.....	5
1.7.4. Objects within objects	5
1.8. Additional work: classes within classes.....	5

1. Introduction

In this lesson, we will do some exercises dealing with a new topic: *Inheritance*, seen this week in the lectures.

1.1. Exercise 1: Inheritance: Class Employee

In this exercise you will implement the first concepts related to inheritance seen in class, for which we use the class Employee and derive from it the class hierarchy shown in the Figure 1. You must create a project called *Exer1* and the *.h and *.cpp files of each of the classes.

Next, the files of the first exercise of the lesson are shown:

```
// File Employee.h

class Employee
{
protected:
    char name[40];
public:
    Employee(char *n=(char*) "") {
        strcpy_s(name,n); }
    void print_position() {
        cout << " Position not defined" << endl;
```

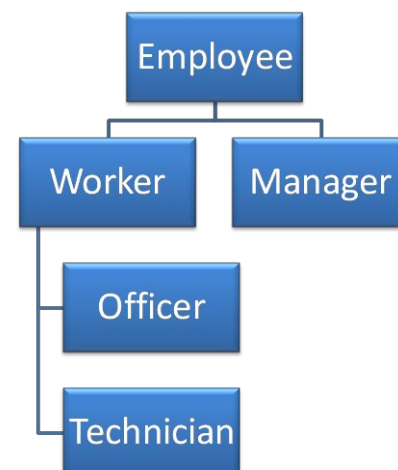


Figure 1 Class hierarchy

```

    }
    char *GetName() { return name; }
};

//In this case, having only a function and since this is very simple, it has been
//decided to implement it as "inline"

class Manager : public Employee
{
public:
    Manager(char *n=(char*)"") { strcpy_s(name,n); }
    void print_position() { cout << " is a manager" << endl; }
};

```

You must implement the rest of the classes so as to be able to write the following piece of code in *main()*:

```

void main()
{
    Employee Rafa((char*)"Rafa");
    Manager Mario((char*)"Mario");
    Worker Anton((char*)"Anton");
    Officer Luis((char*)"Luis");
    Technician Pablo((char*)"Pablo");

    // The type of object determines the function that is called
    cout << "With object names:\n" << endl;
    cout << Rafa.GetName();
    Rafa.print_position();
    cout << Mario.GetName();
    Mario.print_position();
    cout << Anton.GetName();
    Anton.print_position();
    cout << Luis.GetName();
    Luis.print_position();
    cout << Pablo.GetName();
    Pablo.print_position();
}

```

1.2. Exercise 2: Constructor and destructor execution sequences

In this exercise it is important that you can see the object creation sequence, checking the order in which the constructors and destructors are executed. For that, you must include in the constructors the piece of code that prints on the screen a message informing about **which is the constructor** that is being executed. You should do the same **with the destructors**. Notice that the sequence of creation and destruction of objects is inverse, that is, the constructors of the base classes are executed first, while the destructors of the base classes are the last to be run. Use the previous project for this exercise. Use **debugger** or include a **cout** in constructors and destructors.

```

Employee constructor
Employee constructor
Manager constructor

Employee constructor
Worker constructor

Employee constructor
Worker constructor
Officer constructor

Employee constructor
Worker constructor
Technician constructor

With objetc name:
Rafa Position not defined
Mario is a manager
Anton is a worker
Luis is a officer
Pablo is a technician

Technician destructor
Worker destructor

Employee destructor
Officer destructor
Worker destructor

Employee destructor
Worker destructor

Employee destructor
Manager destructor

Employee destructor
Employee destructor
Press any key to continue . . .

```

1.3. Exercise 3: Calling the constructors

Remove the access specifier "**protected**" from the class **Employee** (so it is now private), and modify the constructors for them to call their respective base class, as follows:

```

Manager(char *n=(char*)""): Employee(n) { . . . }

```

1.4. Exercise 4: Adding more variables to the class

Taking the code from the previous exercise, you must add the private variable **double Salary** to the base class Employee, and modify the program in **main()** in order to pass this salary to all objects of the class hierarchy: Official, Technician, Worker, Manager. Also add the function **void prt()** in each class to print the name and salary. For example:

```
void main()
{
    Employee  Rafa((char*)"Rafa", 850); Manager
    Mario((char*)"Mario", 2500); Worker
        Anton((char*)"Anton", 1250);
    Officer   Luis((char*)"Luis", 1500);
    Technician Pablo((char*)"Pablo", 1750);

    Rafa.prt();
    Mario.prt();
    //.. print to console the rest of the objects
}
```

1.5. Exercise 5: Additional work

1.5.1. Adding even more variables

Create next structure containing the name of a specific skill and a score based on performance on that task:

```
struct task
{
    char* task_name;
    double score;
};
```

Now, taking the code from the previous exercise, you must add as a public variable in the base class (employee) an array of “task” structures. This array contains the specific skills of each employee.

```
class Employee
{
    double salary;
protected:
    char name[40];
public:
    task list[20];
    //rest of the code
};
```

Try assigning values from the main function to each of the employees in exercise 1.4, for example:

```
Rafa.list[0].task_name = (char*)"Reparaciones";
Rafa.list[0].score = 10;
```

1.5.2. Form public to protected

To add a little more difficulty to the exercise, you are asked to change the struct array ‘task’ from public to protected, and try to do the same. Note that you will have to do this via a function.

1.5.3. Form protected to private

Do the same as in the previous case, but in this case change the array of 'task' structures from protected to private, and try to do the same thing.

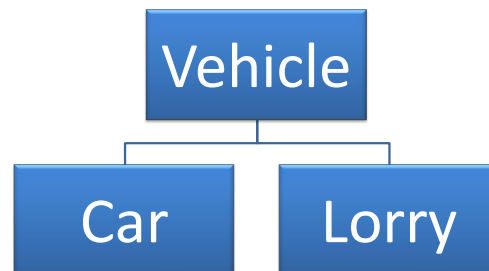
1.5.4. Dynamic structure

Change the array of 'task' structures to pointer, and as in the previous cases assign values from main to each of the employees in exercise 1.4.

```
task *list;
```

1.6. Exercise 5: Vehicle management

This exercise consists of making a program to manage vehicles, which can be Cars or Lorrays. The class diagram is shown in the following figure, where the base class is *Vehicle* and the two derived classes are *Car* and *Lorry*. The base class *Vehicle* has two protected member variables: year of manufacture and the license plate.



Next, the basic structure of the classes is given:

```

//vehicle.h
class Vehicle
{
protected:
    int year;           // year of manufacture
    char plate[20];     // license plate
public:
    Vehicle(int y=2000, char *p=(char*)"AAA1234") {
        year = y;
        strcpy_s(plate,p);
    }
    void display(){cout << "Year:" << year << " Plate:" << plate
<< endl;}
};

class Car : public Vehicle
{
    int hp;             //horsepower
public:
    Car(int y=2000, char *p=(char*)"AAA1234",int horsepower=65);
    void display();
};

class Lorry : public Vehicle
{
    int lmax;           //maximum load in tons
public:
    Lorry(int y=2000, char *p=(char*)"AAA1234",int lm=10);
    void display();
};
  
```

You must implement the rest of the code, both the constructors and the functions so you can run the following program:

```

#include <iostream>
using namespace std;
#include "vehicle.h"

void main()
  
```

```

{
    Car golf(2011, (char*)"BDA2456",150);
    Car astra(2010, (char*)"ABZ2367",110);
    Lorry Pegaso(2009, (char*)"AZT2345",15);
    Lorry Iveco(2012, (char*)"CZT8953",17);

    golf.display();
    astra.display();
    Pegaso.display();
    Iveco.display();

    cout << "End of vehicle list.."<<endl;
}

```

1.7. Additional work

1.7.1. Objects within structures

Create a ‘fleet’ structure of vehicles in the main as follows

```

struct fleet
{
    Car Car_list[10];
    Lorry Lorry_list[10];
};

```

Try to assign values from main a to each of the elements of the vehicle ‘fleet’. How are the functions of the objects within the structure invoked?

1.7.2. Array of structures with objects within the structure

Create an array of ‘fleet’ structures, assign values from the main function and make calls to the functions of the objects contained within the structure.

```

fleet fleet_list[10];

```

1.7.3. Dynamic array of structures with objects within the structure

Do the same as in the previous case but with a dynamic array as follows

```

fleet *fleet_list;

```

1.7.4. Objects within objects

Could an object of type employee be added as a variable of the vehicle class?

```

class Vehicle
{
protected:
    int year;
    Employee e;
    char plate[20];
public:
};

```

1.8. Additional work: classes within classes

Your goal is to build a simulation of a small-scale factory. In this factory:

- **Employees** are assigned to **Products**.
- **Products** can be of two types: **Devices** and **Components**.

- Employees can be of two types: **Engineers** and **Workers**.
- Each class must have relevant attributes, constructors, and methods to interact with other objects.

```

class Product;
class Employee;

//-----
// Employee Hierarchy
class Employee {
protected:
    int ID;
    string name;
    vector<Product*> assignedProducts
public:
    Employee(int id, const string& n) : ID(id), name(n) {}
};

class Engineer : public Employee {
    string specialty;
public:
    Engineer(int id, const string& n, const string& esp)
        : Employee(id, n), specialty(esp) {}
};

class Worker : public Employee {
    int shift;
public:
    Worker(int id, const string& n, int s)
        : Employee(id, n), shift(s) {}
};

//-----
// Product Hierarchy
class Product {
protected:
    int productId;
    string name;
    Employee** assignedEmployees;
    int employeeCount;
    int employeeCapacity;
public:
    Product(int id, const string& n) : productId(id), name(n), employeeCount(0),
        employeeCapacity(5) {
        assignedEmployees = new Employee * [employeeCapacity];
    }
    ~Product() {
        delete[] assignedEmployees;
    }
};

class Device : public Product {
    double voltage;
public:
    Device(int id, const string& n, double v)
        : Product(id, n), voltage(v) {}
};

class Component : public Product {
    string materialType;
public:
    Component(int id, const string& n, const string& type)

```



```
        : Product(id, n), materialType(type) {}
};
```

Implement the following global functions:

- `assignEmployeeToProduct (Employee*, Product*)`: assigns both ways (product to employee and employee to product).
- `showProductionNetwork (vector<Product*>)`: displays products and their assigned employees.
- `showWorkload (Employee**, int)`: displays all employees and how many products they are assigned to.
- `filterEmployeesByProductCount (Employee**, int, int minCount, int& resultCount)`: returns a filtered list of employees assigned to at least `minCount` products.

Your `main()` function must:

- Use a dynamic array of pointers for `Employee* employees` (max 10).
- Use `vector<Product*>` for the list of products.
- Initialize at least **3 employees** and **2 products** at startup.
- Include a **menu** with the following options:
 1. Create a new Engineer.
 2. Create a new Worker.
 3. Create a new Product (Device or Component).
 4. Assign an Employee to a Product.
 5. Show the production network.
 6. Show all employee workloads.
 7. Filter employees by minimum product count.
 8. Exit the program.
- Make sure to free all dynamically allocated memory at the end.

```
void main() {
    int maxEmployees = 10;
    int numEmployees = 0;
    Employee** employees = new Employee * [maxEmployees]; // Dynamic array of
    pointers to Employee
    vector<Product*> products; // Vector of pointers to Product

    // === Initial Employees ===
    employees[numEmployees++] = new Engineer(1001, "Alice", "Electronics");
    employees[numEmployees++] = new Worker(2001, "Bob", 1);
    employees[numEmployees++] = new Worker(2002, "Clara", 2);

    // === Initial Products ===
    Product* p1 = new Device(1, "Tablet Pro", 5.0);
    Product* p2 = new Component(2, "Motherboard", "PCB");
    products.push_back(p1);
    products.push_back(p2);

    assignEmployeeToProduct(employees[0], products[0]);
    // Rest of the code
}
```

Below you have screenshots of the program operation.

<pre> C:\Users\jppuente\Docu x + - □ x === Menu === 1. Create Engineer 2. Create Worker 3. Create Product 4. Assign Employee to Product 5. Show Production Network 6. Show Employee Workload 7. Filter Employees by Product Count 8. Exit Choose an option: 1 Enter Engineer ID: 1005 Enter Engineer Name: Paco Enter Engineer Specialty: Elctric </pre>	<pre> C:\Users\jppuente\Docu x + - □ x === Menu === 1. Create Engineer 2. Create Worker 3. Create Product 4. Assign Employee to Product 5. Show Production Network 6. Show Employee Workload 7. Filter Employees by Product Count 8. Exit Choose an option: 2 Enter Worker ID: 1006 Enter Worker Name: Ana Enter Worker Shift: 3 </pre>
<pre> C:\Users\jppuente\Docu x + - □ x === Menu === 1. Create Engineer 2. Create Worker 3. Create Product 4. Assign Employee to Product 5. Show Production Network 6. Show Employee Workload 7. Filter Employees by Product Count 8. Exit Choose an option: 3 Enter Product ID: 125 Enter Product Name: Capacitor Enter Product Type (Device/Component): Component Enter Material Type for Component: Ceramic </pre>	<pre> C:\Users\jppuente\Docu x + - □ x === Menu === 1. Create Engineer 2. Create Worker 3. Create Product 4. Assign Employee to Product 5. Show Production Network 6. Show Employee Workload 7. Filter Employees by Product Count 8. Exit Choose an option: 3 Enter Product ID: 126 Enter Product Name: Diode Enter Product Type (Device/Component): Device Enter Voltage for Device: 12 </pre>
<pre> C:\Users\jppuente\Docu x + - □ x === Menu === 1. Create Engineer 2. Create Worker 3. Create Product 4. Assign Employee to Product 5. Show Production Network 6. Show Employee Workload 7. Filter Employees by Product Count 8. Exit Choose an option: 4 Enter Employee ID: 1005 Enter Product ID: 125 Employee assigned to product. </pre>	<pre> C:\Users\jppuente\Docu x + - □ x === Menu === 1. Create Engineer 2. Create Worker 3. Create Product 4. Assign Employee to Product 5. Show Production Network 6. Show Employee Workload 7. Filter Employees by Product Count 8. Exit Choose an option: 4 Enter Employee ID: 11111 Enter Product ID: 2222 Employee or Product not found. </pre>
<pre> C:\Users\jppuente\Docu x + - □ x === Menu === 1. Create Engineer 2. Create Worker 3. Create Product 4. Assign Employee to Product 5. Show Production Network 6. Show Employee Workload 7. Filter Employees by Product Count 8. Exit Choose an option: 5 Product: Tablet Pro (ID: 1) - [Employee] Alice (ID: 1001, Products: 1) Product: Motherboard (ID: 2) Product: Capacitor (ID: 125) - [Employee] Paco (ID: 1005, Products: 1) Product: Diode (ID: 126) </pre>	<pre> C:\Users\jppuente\Docu x + - □ x === Menu === 1. Create Engineer 2. Create Worker 3. Create Product 4. Assign Employee to Product 5. Show Production Network 6. Show Employee Workload 7. Filter Employees by Product Count 8. Exit Choose an option: 6 [Employee] Alice (ID: 1001, Products: 1) [Employee] Bob (ID: 2001, Products: 0) [Employee] Clara (ID: 2002, Products: 0) [Employee] Paco (ID: 1005, Products: 1) [Employee] Ana (ID: 1006, Products: 0) </pre>

<pre> C:\Users\jppuente\Docu x + v - □ x === Menu === 1. Create Engineer 2. Create Worker 3. Create Product 4. Assign Employee to Product 5. Show Production Network 6. Show Employee Workload 7. Filter Employees by Product Count 8. Exit Choose an option: 7 Enter minimum number of products: 1 Employees with at least 1 products: [Employee] Alice (ID: 1001, Products: 1) [Employee] Paco (ID: 1005, Products: 1) </pre>	
---	--